Week 2 Reading: Steady-State MLPs

1. Three-component steady-state CSTR outputs

For consecutive first-order reactions A → B → C, setting each derivative to zero lets us solve in the order A, B, then C:

$$C_A = \frac{C_{\mathrm{Af}}}{1 + k_1\tau}$$

$$C_B = \frac{k_1\tau C_{\mathrm{Af}}}{(1 + k_1\tau)(1 + k_2\tau)}$$

$$C_C = k_2\tau C_B$$

At 330 K, the first and second reaction rates are 0.2/min and 0.1/min. With 5 min residence time and 1.2 mol/L feed, the three concentrations are [0.6, 0.4, 0.2] mol/L. Their sum is 1.2. These are the labels for one steady-state training row.

The input is [T, τ, C_Af] and the output is [C_A, C_B, C_C]. Each row is one steady operating condition.

2. Count parameters before training

A dense layer has one weight for each input-output pair and one bias per output. The steady MLP is 3 → 32 → 32 → 3:

Layer	Count
First hidden layer	3 × 32 + 32 = 128
Second hidden layer	32 × 32 + 32 = 1056
Output layer	32 × 3 + 3 = 99
Total	**1283**

$$h_1 = \mathrm{ReLU}\left(W_1 \tilde{\mathrm{x}} + b_1\right)$$
$$h_2 = \mathrm{ReLU}\left(W_2 h_1 + b_2\right)$$
$$\hat{z} = W_3 h_2 + b_3$$

ReLU applies max(0, value) to each hidden pre-activation. A linear output can return negative concentrations; nonnegativity and concentration-sum consistency must therefore be evaluated.

Computing column equations as a batch matrix

The equations use column inputs. If batch code stores X as (N,3) rows, the first layer is `X @ W1.T + b1`, producing (N,32). Transposing the weights implements the same weighted sums with row storage; the 32 biases are applied to every data row.

Array	Shape	Axis meanings
X	N×3	data rows, input variables
W1.T	3×32	input variables, hidden neurons
X @ W1.T	N×32	data rows, hidden neurons
b1	32	one bias per neuron

Take input rows [1,0,−1] and [0,1,1], first-neuron weights [1,0,0], and bias 0.2. The first output column contains $1\times1+0\times0-1\times0+0.2=1.2$ and $0\times1+1\times0+1\times0+0.2=0.2$. The other 31 columns repeat this calculation with their own neuron weights.

ReLU then acts on each matrix entry separately. This operation does not mix different data rows.

3. Output scaling and componentwise loss weights

Use the declared bounds for input scaling. For output standardization, compute the mean and standard deviation from training data only; restore physical units before reporting concentration errors.

$$\tilde{x}_j = 2\frac{x_j - l_j}{u_j - l_j} - 1$$

$$z_j = \frac{y_j - \mu_j}{s_j}, \quad \hat{y}_j = \mu_j + s_j \hat{z}_j$$

$$L = \frac{1}{3N} \sum_{n=1}^{N} \sum_{j=1}^{3} \left(\hat{z}_{n,j} - z_{n,j} \right)^2$$

A raw squared error is weighted by the inverse squared training standard deviation. With standard deviations approximately 0.283 for A and 0.110 for B, the same raw squared error receives about 6.62 times the weight for B compared with A.

Compare component test RMSE and the concentration-sum residual with and without output scaling. Check the averaging convention in the official MSE definition.

4. Worked backpropagation and parameter update

Use a 3 → 2 → 3 network with `W1 = [[1, 0, 0], [0, 0, 1]]`, `W2 = [[1, 0], [0, 1], [1, 1]]`, and zero biases. Input [1, 0, −1] gives hidden pre-activation [1, −1], ReLU output [1, 0], and prediction [1, 0, 1]. Against target [0, 1, 0], the error is [1, −1, 1].

$$\delta_2 = e, \nabla_{W_2} L = eh^T = \begin{bmatrix} 1 & 0 \\ -1 & 0 \\ 1 & 0 \end{bmatrix}$$

$$\delta_1 = \left({W_2}^T e \right) \odot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

$$\nabla_{W_1} L = \delta_1 x^T = \begin{bmatrix} 2 & 0 & -2 \\ 0 & 0 & 0 \end{bmatrix}$$

The output-weight gradient is error × hidden activation transposed. The hidden error uses the output weights and then the ReLU derivative; a negative pre-activation blocks that component's gradient.

Bias gradients are the corresponding error vectors. Updating both weights and biases with learning rate 0.1 gives the next prediction [0.26, 0.14, 0.26]. The loss ½‖prediction − target‖² falls from 1.5 to 0.4374.

PyTorch autograd performs this chain-rule propagation automatically; the notebook checks the hand calculation.

The chain rule for one weight

The hand calculation uses $L=\frac{1}{2}\sum e_j^2$ with $e_j=\hat{y}_j-y_j$. Its derivative with respect to prediction $\hat{y}_j$ is e_j. Increasing output weight $W_{2,ji}$ changes $\hat{y}_j$ at rate h_i.

Derivative path	Factors	Value in the example
$W_{2,A1} \to \hat{y}_A \to L$	$e_A \times h_1$	$1 \times 1 = 1$
$W_{2,B1} \to \hat{y}_B \to L$	$e_B \times h_1$	$-1 \times 1 = -1$
$h_1 \to$ three outputs $\to L$	sum of weight×error	$1 \times 1 + 0 \times (-1) + 1 \times 1 = 2$
$W_{1,13} \to a_1 \to h_1 \to L$	$x_3 \times \mathrm{ReLU}'(a_1) \times 2$	$(-1) \times 1 \times 2 = -2$

ReLU has derivative 1 for $a>0$ and 0 for $a<0$; this example avoids $a=0$. A bias enters the weighted sum with coefficient 1, so its derivative uses 1 instead of an input value.

δ denotes the loss derivative with respect to a layer's pre-activation; $\odot$ multiplies matching components. For the actual mean-squared loss, the output derivative is $2e/(3N)$, with a different factor from the hand calculation's half squared sum.

The second forward pass after updating parameters

Update both weights and biases with learning rate 0.1. Subtracting $0.1\times[2,0,-2]$ from the first hidden weight $[1,0,0]$ gives $[0.8,0,0.2]$; its bias becomes -0.2.

Intermediate value	Updated calculation
First hidden pre-activation	$0.8\times1+0\times0+0.2\times(-1)-0.2=0.4$
Second hidden pre-activation	-1
ReLU output	$[0.4,0]$
Outputs A and C	$0.9\times0.4-0.1=0.26$
Output B	$0.1\times0.4+0.1=0.14$

The new error is $[0.26,-0.86,0.26]$. Loss falls to $(0.0676+0.7396+0.0676)/2=0.4374$, but B remains below its target 1. One decrease does not complete training.

5. Read the training loop as a calculation

Each batch computes predictions and a loss, clears previously accumulated parameter gradients, differentiates the current loss, and applies an optimizer update:

```
prediction = model(inputs)
loss = loss_function(prediction, targets)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

`loss.backward()` computes gradients; `optimizer.step()` changes parameters. Without clearing gradients, successive backward passes accumulate them.

Evaluate validation loss after an epoch and retain the weights at its minimum. Keep the test set out of parameter fitting and model selection. Compute prediction metrics after undoing output standardization.

Original explanations: PyTorch training loop and model construction.

Values and changes inside the training loop

```
prediction = model(inputs)
loss = loss_function(prediction, targets)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

Line	Value produced or changed
model(inputs)	Batch predictions with current weights
loss_function	Prediction errors reduced to a scalar loss
zero_grad	Previous batch gradients cleared
backward	Current loss gradients for weights and biases
step	Parameters updated using gradients and optimizer state

The hand calculation uses gradient descent; the lab uses Adam. Adam also uses gradient history, so its numerical update is not the same subtraction by 0.1 times the gradient. To assess the updated parameters, run a new forward pass after step.

Adam’s first update and the meaning of a batch

A batch is a group of rows used for one loss and gradient calculation. Splitting 400 rows into batches of 64 gives six full batches and a final batch of 16, hence seven updates per epoch. An epoch is one pass through the training data.

Adam accumulates a gradient average m and a squared-gradient average v. Start with $m=v=0$ and consider one weight with $g=2$, $\beta_1=0.9$ and $\beta_2=0.999$.

Calculation	First-step value
$m=0.9m+0.1g$	0.2
$v=0.999v+0.001g^2$	0.004
Correction for initialization at zero	$\hat{m}=0.2/(1-0.9)=2$; $\hat{v}=0.004/(1-0.999)=4$
Update magnitude at learning rate 0.001	$0.001\times2/(\sqrt{4}+10^{-8})\approx0.001$

Plain gradient descent would subtract $0.001\times2=0.002$. Adam scales the step using accumulated gradient magnitudes, so equal learning rates need not give equal updates. Here $\varepsilon=10^{-8}$ prevents a zero denominator.

PyTorch Adam update definition

6. Diagnose the held-out predictions

Suppose a reference row is [0.6, 0.4, 0.2] mol/L and the predictor returns [0.62, 0.39, 0.19] mol/L.

The component errors are [0.02, −0.01, −0.01] mol/L. Both sums equal 1.2, so the concentration-sum residual is zero. Correct total concentration does not make each component exact.

Check: which quantities should a test report include, and which data may be used to choose the model?

Answer check: report A/B/C RMSE in physical units, concentration-sum residual, and minimum predicted concentration. Fit on train, choose with validation, and leave test out of both steps.

Check the same metrics in the Week 2 core lab.

From standardized outputs to concentrations and errors

For a one-component calculation, take training mean μ=0.4 mol/L and standard deviation s=0.1 mol/L. Target concentration 0.45 becomes z=(0.45−0.4)/0.1=0.5.

Quantity	Calculation
Standardized network prediction	$\hat{z}$=0.3
Restore concentration	0.4+0.1×0.3=0.43 mol/L
Standardized error	0.3−0.5=−0.2
Concentration error	0.43−0.45=−0.02 mol/L

```
prediction = y_mean + y_scale * standardized_prediction
component_rmse = np.sqrt(((prediction - truth)**2).mean(axis=0))
sum_residual = prediction.sum(axis=1) - feed
```

The first line applies each column’s mean and scale. axis=0 measures accuracy by component; axis=1 sums the three components at one condition. Swapping these axes computes a different quantity.